



Detecting Smishing Messages Using BERT and Advanced NLP Techniques

Ankit Kumar Jain¹ · Kamaljeet Kaur¹ · Naveen Kumar Gupta² · Ankit Khare³

Received: 29 June 2024 / Accepted: 12 November 2024

© The Author(s), under exclusive licence to Springer Nature Singapore Pte Ltd. 2025

Abstract

The study focuses on the detection of smishing attacks, a variant of phishing that exploits text messages to dupe mobile users into revealing private information, downloading malware, or clicking malicious links. Despite the existence of various approaches, there remains a gap in timely smishing detection and the ability to effectively distinguish between smishing and legitimate SMS messages. To address these issues our approach focuses on detecting smishing by identifying key features and using advanced models like BERT, combined with different loss functions. Through an exploration of embeddings including TF-IDF and BERT, we developed a classifier model that achieved satisfactory accuracy. Notably, BERT emerged as the most effective, adept at capturing contextual nuances and word associations. To further enhance accuracy, we transformed the original tri-class challenge into dual binary classification tasks, employing two multi-layer neural networks. Our research highlights the application of advanced Natural Language Processing (NLP) techniques such as tokenization and self-attention mechanisms for precise message categorization into smishing, ham, or spam groups. The logistic regression classifier achieved the highest accuracy of 94% on the 4:1 ratio dataset. In summary, our approach enhances security by accurately detecting smishing through a combination of feature extraction, advanced NLP techniques, and effective classification models.

Keywords Smishing · Phishing · SMS · Malware · URL · BERT

Introduction

Phishing is a kind of cyber fraud where attackers aim to impersonate respectable companies and contact individuals using emails, SMS, social media, gaming, productivity, sports, dating, ecommerce, and mobile applications to get sensitive information from consumers or persuade them to install malware on their devices [1]. Smishing is a form of phishing that uses a short messaging service (SMS), a prevalent communication method. Attackers initiate smishing by sending malicious text messages to smartphone users. Figure 1 presents the Schematic Sequence of Smishing Attack.

This attack typically includes a URL, phone number, or email address, along with content urging the recipient to reach out to the attacker. When the mobile user responds either through phone or email, the attacker instructs them to download a harmful application. Mistakenly believing it's legitimate, the user installs this software, allowing the attacker to gain access to their device and harvest personal and financial data. This information can subsequently be used to inflict significant financial damage on the user [2].

According to SMS marketing statistics, 90% of people respond to a text message within three minutes, and 99% of SMS are opened within twenty minutes, showcasing its immediacy [3]. Furthermore, text messages boost a 98% response rate, vastly outperforming emails, which have a mere 20% response rate [4]. As mobile connections have proliferated, short text messaging has emerged as the most efficient mode of real-time communication. Although various messaging services exist today, SMS remains the most global communication method [24]. This is attributed to its universal compatibility, as every mobile phone globally comes equipped with SMS capability. Owing to these

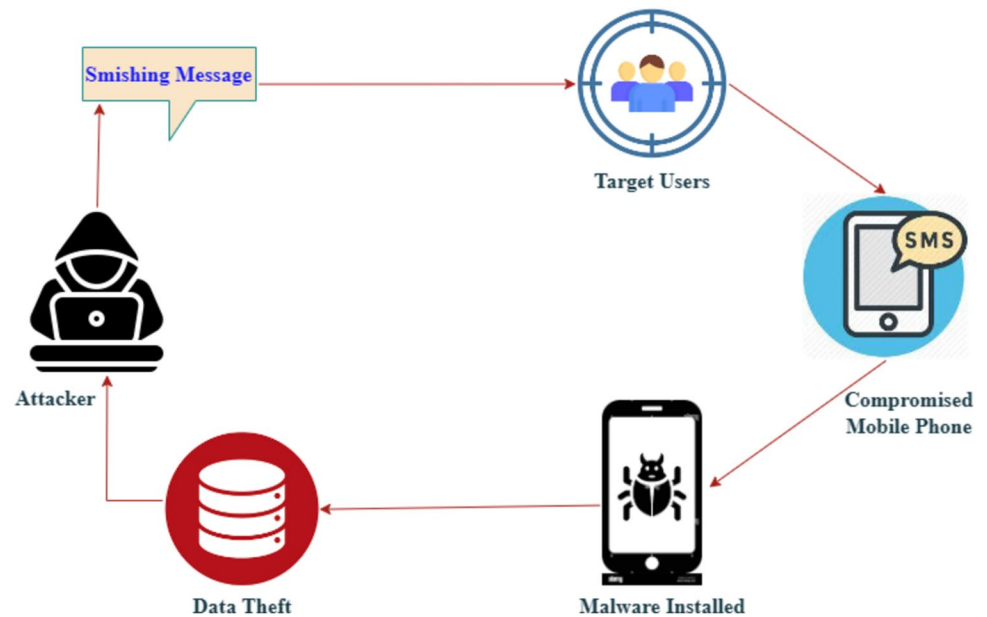
✉ Ankit Kumar Jain
ankit.jain2407@gmail.com

¹ National Institute of Technology Kurukshetra, Kurukshetra, India

² Dr. B.R. Ambedkar National Institute of Technology, Jalandhar, India

³ Chitkara University Institute of Engineering and Technology, Chitkara University, Rajpura, Punjab, India

Fig. 1 Schematic Sequence of Smishing Attack



advantages, SMS has evolved into a powerful tool for global marketing and communication. It is frequently employed in sectors like marketing, retail, healthcare, and banking for customer engagement, corporate updates, and reminders [5].

Increasing public awareness of the risks associated with smishing and equipping people with defenses against such attacks is crucial. This paper aims to classify messages as either smishing or ham using state-of-the-art Natural Language Processing text classification algorithms such as NB, SVM, RF, and neural network-based [1, 5]. The transformer model is utilized to derive a fixed-dimension embedding for a message. This embedding can then serve as input for various machine-learning classifiers to facilitate message classification. The methodology incorporates tokenization, self-attention mechanisms, and machine learning classifiers, including but not limited to neural networks, to achieve impressive accuracy in text classification. By creating an initiative against smishing, we can raise awareness and provide individuals with the necessary tools to protect themselves from these attacks [6].

The Internet is always expanding. People in the modern world are required to utilize internet-connected devices. Because of their ease, online purchases have also grown in popularity. In such an environment, there is always anxiety about cybersecurity. Anybody who has access to a person's net banking information can take the money out of the account even if they are not the legitimate owner of the funds. This is accomplished using fraudulent emails, SMS, and other methods [7]. The stolen information may be used to add new phone numbers to a customer's account, giving the attackers complete control of the account. This happened in the United States, where \$100,000 was taken in two weeks from 17 ATMs in Ohio, Michigan, and Illinois [8]. SMS

has a 98% open rate against 45% for emails. SMS also has a 45% response rate, which makes it an effective method for fraudsters to steal information [9]. Fintech businesses have encouraged individuals to embrace digital payment methods as a result of recent events such as COVID-19, the introduction of UPI, demonetization, and the usage of digital wallets. Most transactions are now settled digitally, increasing the risk of banking fraud. Figure 2 shows an increase in mobile banking Trojans detected in Q2-2023 [25]. In 2018, 17% of business related links received on mobile in the form of smishing attacks and in 2022, 27.6% of users clicked on smishing links and fell victim to mobile-based fraud [10]. According to a UN study, phishing attacks increased by 350% during the epidemic due to the online use of applications to access service providers such as Zomato, Amazon, Urban Company, Ola, Uber, Swiggy, etc. Attackers might pose as any of the online services [6].

This brutal attack is accomplished not only to steal user's money but also to get access to their contacts, photographs, and other data recorded on their cell phones. The tiny display size on the majority of mobile devices makes it challenging for consumers to independently check the contents of links that appear suspicious, making them exposed to smishing attacks. Figure 3 presents some examples of smishing messages containing a malicious phone number to call back, and other with malicious URL to fill delivery preferences and to validate account [11]. Harmful SMS messages with malicious URLs have resulted in malware attacks on phones, stealing user data and financial information. This method additionally makes it simple for hackers to access the data on users' phones. A survey found that 68% of mobile phone users are impacted by SMS spam, with young adults being the most at risk [5]. Raising awareness about smishing

Fig. 2 Number of mobile banking Trojan statistics from Q2-2022 to Q2-2023

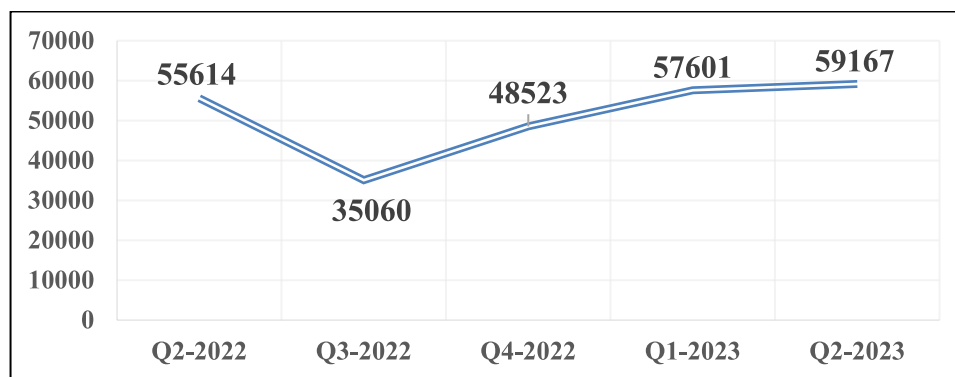
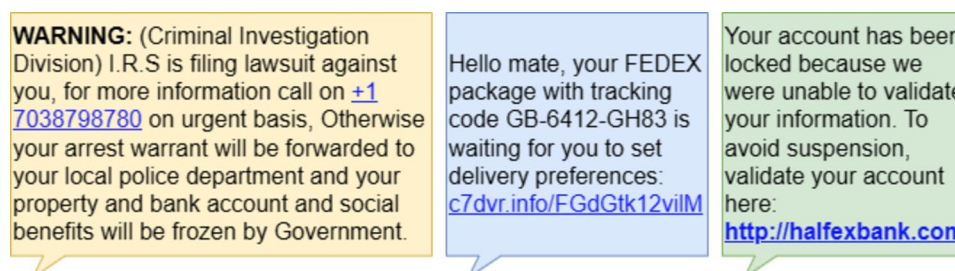


Fig. 3 Examples of Smishing messages



threats and educating the public on protective measures is crucial. A dedicated smishing paper can offer essential tools and insights to defend against these threats.

The following are the key contributions of the proposed method:

1. Development of an advanced NLP model that precisely distinguishes between ham (legitimate), spam, and smishing messages.
2. Introduction of a new detection framework that combines BERT with machine learning methods to enhance smishing detection accuracy.
3. Evaluation of the effectiveness of different Train: Test (2:1 and 4:1) ratios on the proposed model.

The rest of the paper is structured as follows: Sect. [Related Works](#) discusses related work, Sect. [Proposed Work](#) presents the proposed work, Sect. [Experiment](#) outlines the experimental setup, and finally, Sect. [Conclusion and Future Directions](#) offers a conclusion which summarizes the main points of the paper.

Related Works

The studies primarily focus on the identification and prevention of smishing, a deceptive SMS-based phishing method used to extract personal information. Numerous discussed

research efforts aim to develop strategies and tools to counteract this growing threat.

A lot of effort has been invested in the field of machine learning to enhance user security. A variety of significant features, such as the presence of URLs (especially short URLs), the downloading of APK files (whether prompted or unprompted), keywords commonly used in smishing messages, cellphone numbers, currency symbols, and more have been identified. Neural networks have detected smishing communications with remarkable accuracy. Various techniques, including RIPPER (a rule-based learning algorithm), SVM, and Random Forest, have been employed to identify smishing messages, each achieving different levels of accuracy [5]. Mishra et al. [1] introduced an efficient neural network-based model for smishing detection, known as the smishing detector. Social engineering is a tactic used in SMS phishing known as smishing, which deceives targets into divulging personal information. The suggested model recognizes smishing messages using feature extraction and classification methods. Key characteristics from the SMS message, including the sender's phone number, the message's content, and any embedded URLs, are extracted using the feature extraction approach. A neural network is used in the classification approach to determine if the communication is authentic or smishing. Oswald et al. [2] presented Spot Spam, an intention analysis-driven technique for detecting SMS spam that uses BERT embeddings. Pre-trained BERT is a language model that can provide high-quality word embeddings that accurately represent a word's meaning and context. The

suggested technique encrypts SMS messages using BERT embeddings and looks for patterns that point to spam. Furthermore, a smartphone application for real-time smishing detection using rule-based techniques was described by Akande et al. [5]. The program analyzes the content of SMS messages using a rule-based methodology to spot trends that point to smishing attempts. The guidelines are meant to catch several aspects of smishing attacks, such as demands for sensitive information, threatening or urgent language, and dubious URLs. Users may also get a report on the program's message analysis, highlighting the particular criteria that led to the smishing detection.

Moreover, Mishra et al.'s methodology [12] uses URL behavior analysis and SMS content analysis to recognize and stop smishing attacks. Natural language processing methods are used to evaluate the SMS text to find keywords and trends suggesting smishing attacks. The study of URL activity involves observing the actions of the embedded URLs in the message, including page loads, pop-ups, and redirection. The model learns from previous assaults and adjusts against new kinds of attacks using machine learning methods. With a precision of 98.4% and a recall of 97.7%, the testing findings demonstrate that it achieves a high degree of accuracy and efficiency in identifying smishing assaults. Jain et al. [13] provide a unique method for identifying SMS phishing assaults, often known as smishing attacks. There are three primary phases in the suggested procedure. The SMS message is pre-processed to extract essential properties and delete unnecessary information. Second, the message content is examined using machine learning techniques to find patterns and phrases suggestive of smishing attacks. Third, a specially designed URL analysis tool is used to examine the embedded URLs in the message and find any URLs that could be harmful or suspicious. The suggested technique offers several benefits over current approaches, including minimal computing cost and the ability to identify novel and developing smishing attacks.

Additionally, Jain et al. [14] implemented a rule-based smishing SMS detection approach that classifies malicious SMS based on nine rules. The approach achieved an accuracy rate of 99% but has a false negative rate of 7% using RIPPER. Salman et al. [15] introduces a new SMS dataset consisting of over 68,000 messages, making it the largest publicly available SMS spam dataset to date. The dataset includes both legitimate (ham) and spam messages, providing a valuable resource for future research in the area. Jain et al. [16] developed a machine learning-based approach to detect and filter SMS spam, identifying ten key features that effectively differentiate spam from ham messages. Using the Random Forest algorithm, method achieved a 96.5% true positive rate and a 1.02% false positive rate. Furthermore, Akande et al. [17] introduced SMSPROTECT, which is a mobile automated smishing detection tool.

It uses machine learning methods to identify and stop SMS-based phishing attempts.

A thorough review of Smishing Prediction Systems (SPS), which are intended to identify and avert SMS-based phishing attempts, was provided by Smitha et al. [7]. The authors summarize the state of the art in this field and look at various approaches and procedures used in various SPSs. Santosh et al. [18] present a comprehensive survey of various phishing detection and mitigation techniques. Ghourabi et al. [6] presented a novel method based on transformer-based embedding and ensemble learning to increase the precision of spam categorization and detection. The three steps of the suggested approach are feature extraction, classification, and pre-processing. Spam communications are cleaned up during pre-processing by eliminating stop words, URLs, and special characters. Then, using a transformer-based paradigm, the message is tokenized while maintaining its embeddings. Embeddings are used in the feature extraction stage to extract word frequency, message length, and emotion score characteristics. Each message's feature vector is then created by combining these characteristics. Communications are categorized as spam or non-spam during the classification phase using an ensemble learning technique. Multiple classifiers, including logistic regression, random forests, and support vector machines, are included in ensemble models. Message feature vectors train classifiers, and the outputs are merged by majority vote.

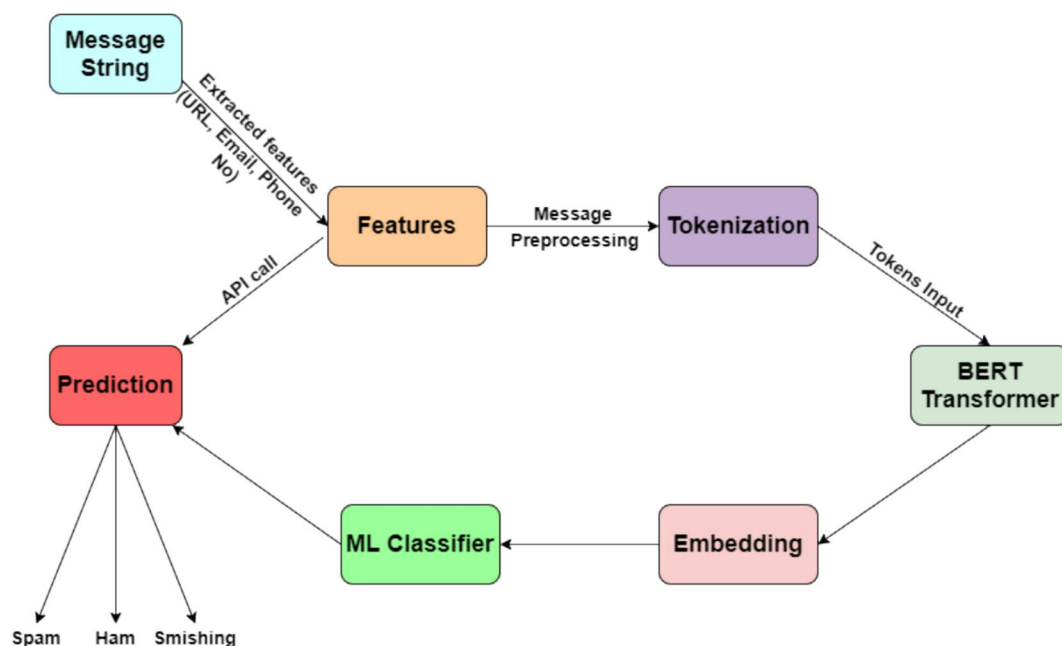
Table 1 comprises the comparison of existing smishing detection methods based upon methodology, dataset used and features. It shows smishing detection is the subject of recent research that uses novel approaches like rule-based mobile apps, BERT embeddings, neural networks, and Transformer-based models. Mishra et al. used feature extraction and classification in their neural network model, whereas Oswald et al. utilized BERT for spam detection. Rule-based techniques for real-time detection were provided by Akande et al. [17], ensemble learning and Transformer-based embedding were merged by Ghourabi et al. [6] to achieve better spam categorization. Together, these pieces tend toward more complex and precise real-time smishing detection techniques. The proposed work aims to study smishing using text classification and transformers, extracting key features like phone numbers and URLs from SMS for optimized classification. The research work evaluates various transformers, tokenizers, and embeddings to determine the best classifier model and develop a user-friendly interface for its application.

Proposed Work

The main motive of this technique is to classify SMS into three main categories spam, ham, and smishing. To achieve the desired goal firstly we have extracted valuable and

Table 1 Comparison of existing smishing detection techniques

Author and Year	Methodology	Dataset used	Features
Mishra et al. (2022) [1]	Smishing detector: neural network-based	Almeida spam dataset	Phone number, URL, message content
Oswald et al., (2022) [2]	Spot Spam, an intention analysis-driven uses BERT embedder	SMS spam dataset v.1, Grumble text Website, NUS SMS Corpus (NSC) Caroline Tag's PhD Thesis	Selected 13 labels based on intention
Akande et al., (2022) [5]	Smartphone application for real-time smishing detection using rule-based	SMS spam dataset v.1	Target mainly demands for sensitive information, threatening or urgent language, and dubious URLs
Mishra et al., (2020) [12]	Natural language processing	Almeida spam dataset	URL behavior analysis and SMS content analysis
Jain et al. (2022) [13]	Machine learning	SMS spam dataset v.1	Sender's phone number, message content, and embedded URLs
Akande et al., (2023) [17]	SMSPROTECT: NLP and Machine learning	SMS spam dataset v.1	Text and metadata features such sender information, message content, and delivery time
Ghourabi et al. (2022) [6]	Transformer-based embedding and ensemble learning	Almeida spam dataset	Word frequency, message length, and emotion score
Das et al. (2024) [19]	Hybrid model (LSTM+DCNN) and optimization ROA	UCI	Automatic feature extraction using the pooling layer
Seo et al. (2024) [20]	Deep Learning	KISA (Korean Internet and Security Agency)	URL position, length of text, symbols, tokens, space

**Fig. 4** Flow diagram of the proposed model

effective characteristics from SMS strings, such as phone number, website, and email address, to aid in classification. Investigating various transformers, tokenizers, and pre-trained embeddings to obtain a decent feature space that can be utilized as an input to our classifier model. Figure 4 illustrates the proposed method. Beginning with

extracting the features next is preprocessing the message and content followed by tokenization and tokens inputs will be passed to transformer and embedder. Furthermore, various machine learning and deep learning models have been used for the task of determining the optimal model. There have been several technological measures performed

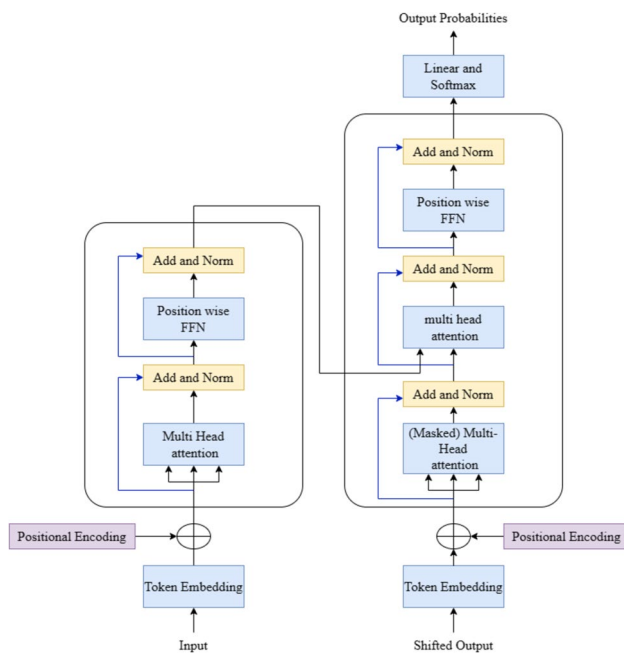


Fig. 5 Transformer Model Architecture

to construct this work. The detail about the transformer is explained below:

Transformers

In Fig. 5, the transformer model makes use of self-attention mechanisms and embedding algorithms. The mechanism of attention instructs the computer regarding which portions of the input should receive higher and lower levels of focus and this concept of focus has been derived from the idea of people's priority, as they give differing amounts of attention to different portions of a sentence or paragraph. In other words, some input components stimulate readers more than others, and those input components have strong significance for the model. While processing a model, self-attention demonstrates how closely connected one input position is to every other input position. Text words are transformed into vectors of continuous real numbers via embedding, which is easier for models to handle.

Components of the Transformer

- **Embedding Layer:** The Embedding Layer is crucial for turning discrete words into continuous vector representations. Every unique word in the input gets mapped to a vector in this high-dimensional space through a lookup table, capturing the semantic nuances of the word. However, Transformers don't inherently understand the sequence or order of words. To compensate for this, positional encodings are added to the word

embeddings. This ensures that the model recognizes and considers the relative or absolute position of words within a sequence.

- **Multi-Head Self-Attention Mechanism:** At the heart of the Transformer model, there is the Multi-Head Self-Attention Mechanism. It allows the model to focus on different parts of the input simultaneously. For each word, three vectors are computed: Query (Q), Key (K), and Value (V). These vectors respectively represent the current context of the word, the context it needs to match, and its actual content. An attention score is then calculated using Q and K to decide how much emphasis a word should place on another. Instead of just one attention mechanism, the model employs multiple heads in parallel. Each of these heads has its set of Q, K, and V matrices, allowing the model to capture multiple relationships and contexts between words.
- **Feed-Forward Neural Networks:** Following the attention mechanism is the Feed-Forward Neural Network layer. Every position in the input sequence undergoes a transformation through this neural network independent of the others. This component offers the Transformer an additional opportunity to transform and refine the information captured by the attention mechanism, allowing the model to capture more intricate relationships and patterns in the data.
- **Residual Connection:** Deep networks like Transformers can be challenging to train. The Residual Connection is a crucial design choice to tackle this. After each major operation either self-attention or the feed-forward step, the original input is added back to its output. These shortcut connections not only promote training stability but also help in alleviating the vanishing gradient problem commonly encountered in deep networks.
- **Positional Encoding:** These encodings introduce information about the position of words in a sentence, ensuring the model understands the order of words.
- **Normalization and Feed Forward Layers:** Post attention mechanism, the outputs are normalized to ensure activations stay within a certain range. This Normalization layer promotes stable and consistent behavior during training. Following normalization, the data is passed through another feed-forward neural network. This process involves another round of transformations, refining the outputs, which then undergo a second residual connection and normalization.
- **Stacking:** A single Transformer block can capture a certain level of patterns and relationships in the data. To harness deeper and more complex patterns, several blocks are stacked and this stacking increases the model's depth and capacity, allowing it to handle a wide variety of intricate tasks. Both the encoder and decoder sections of the Transformer model are made up of such stacked blocks.

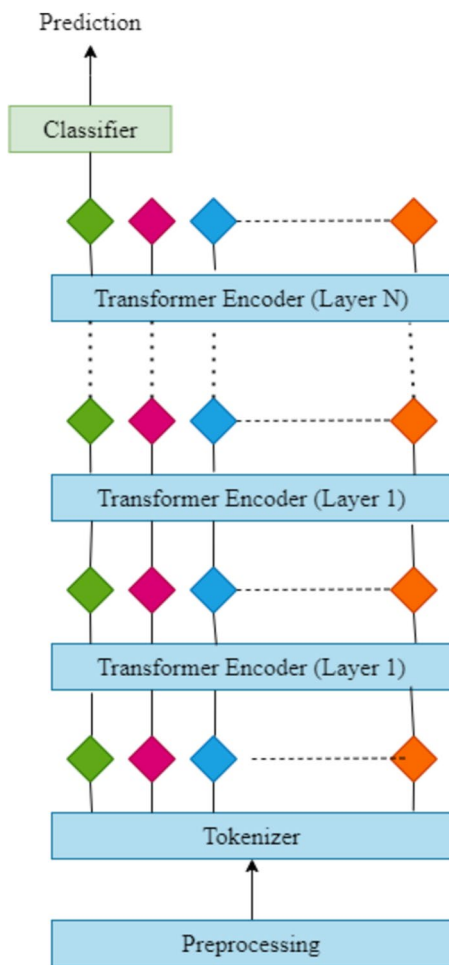


Fig. 6 High-Level Overview of Transformer Text Classification Model

- **Encoder:** The Encoder processes the input sequence to create a dense, context-rich intermediate representation. Starting with embeddings of the input words enhanced with positional encodings, these representations undergo a series of transformations through the stacked Transformer blocks. Each block refines the representation, ensuring the output holds rich contextual information about the entire input sequence.
- **Decoder:** The decoder takes the encoder's output as input. It combines an embedding layer, multiple transformer blocks, and an additional multi-head attention layer to attend to the encoder's output, and the final layer is followed by a Softmax to produce a prediction.

Figure 6 comprises the text classification model which is a kind of neural network design used for text summarization and other NLP applications. It is built from a stack of encoders and decoders coupled with attentional processes. The self-attention layer and the feed-forward

neural network layer are the two layers that make up each encoder. Each word in a phrase may attend to every other word in the sentence thanks to the self-attention layer, which produces a contextual representation of the input sequence. This contextual representation is transformed nonlinearly by the feed-forward neural network layer. With the inclusion of an encoder-decoder attention layer, the decoder and encoder are comparable to one another. This layer enhances the output quality by enabling the decoder to focus on the necessary portions of the encoder's output. During training, the model evolves to give certain words in the sequence of inputs varying weights based on how important they are for predicting the final result. When compared to conventional sequence-to-sequence models, this attention mechanism helps the model concentrate on the input sequence's most important segments, improving performance.

Proposed Algorithm

The proposed method comprises three main components: (i) BERT embedding for URLs, (ii) Extraction of URLs, phone numbers, and email addresses, and (iii) Message classification algorithm.

- BERT Embedding Method:** The BERT embedding method includes the pre-trained Bidirectional Encoder Representations from Transformers (BERT) model to acquire contextualized embeddings for input text. Instead of using traditional static word embeddings, BERT captures the context and semantics of words based on their surrounding context in a given sentence.

The model normally includes the following steps:

- Tokenization:** Let s be the input string. We tokenize s using BERT tokenizer to obtain a sequence of tokens $\mathbf{T} = \{t_1, t_2, \dots, t_n\}$.

$$\mathbf{T} = \text{Tokenizer}(s) \quad (1)$$

- Conversion to Input IDs and Attention Mask:** Next, text input needs to be converted into a format suitable for the model. This typically involves two main components: Input IDs and Attention Mask. We convert the tokens to corresponding input IDs and attention masks.

$$\mathbf{I} = \{\text{id}(t_1), \text{id}(t_2), \dots, \text{id}(t_n)\} \quad (2)$$

$$\mathbf{A} = \{a_1, a_2, \dots, a_n\}$$

where $a_i = 1$ if t_i is not a padding token, and 0 otherwise.

- c) Passing through the BERT Model: We feed the input IDs and attention mask into the BERT model to obtain the hidden states.

$$\mathbf{H} = \text{BERT}(\mathbf{I}, \mathbf{A}) \quad (3)$$

Here, $\mathbf{H}$ is the sequence of hidden states for each token, including the special [CLS] token.

- d) Retrieve the Last Hidden State of [CLS] Token: The last hidden state corresponding to the [CLS] token is given below, which can be considered as a representation of the entire input sequence.

$$\mathbf{h}_{CLS} = \mathbf{H}_{[CLS]} \quad (4)$$

Here, $\mathbf{H}_{[CLS]}$ represents the hidden state of the [CLS] token.

- e) Obtain the 768-Dimensional Embedding: A 768-dimensional encoding typically refers to a vector representation of data in a 768-dimensional vector space. In the context of natural language processing (NLP), it often refers to the output of certain neural network models, such as BERT. These embeddings typically have a fixed dimensionality, such as 768 dimensions. Each dimension in the vector captures certain semantic or syntactic

aspects of the token in the context of the surrounding words. These 768-dimensional embeddings are useful for various NLP tasks, including text classification, named entity recognition, and question answering, among others.

The hidden state of the [CLS] token is already a 768-dimensional embedding vector. Thus, the 768-dimensional embedding is.

$$\mathbf{e}_{CLS} = \mathbf{h}_{CLS} \quad (5)$$

- f) Output the Embedding: Finally, the output of the algorithm is the 768-dimensional embedding of the [CLS] token.

$$\text{Output} = \mathbf{e}_{CLS} \quad (6)$$

- (ii) Extraction of URL, Phone Numbers and Email Addresses: In algorithm 1, “URL, Phone Number, and Email Extract” procedure scans an input string to identify URLs, phone numbers, and email addresses. Using Python’s “re” library, it employs specific patterns for each type. On execution, it returns lists of detected URLs, phone numbers, and emails from the input string.

Algorithm 1: URL, Phone Number and Email Extract

Inputs: Input String
Output: 3-dimensional bool array

1. Imports the **re** library for regular expression operations.
2. Defines a function **find_features(input_string)** that takes an input string as its argument.
3. Defines regular expression patterns for URLs, phone numbers, and emails.
4. Uses the **re.findall()** function to find all occurrences of URLs, phone numbers, and emails in the input string.
5. Returns the extracted URLs, phone numbers, and emails as tuples

- (iii) Message classification Algorithm: In algorithm 2, given an input message ‘m’, we extracted the components, like URL, phone number, and email, using Algorithm 1. After tokenizing ‘m’, we obtain a set

‘T’. Using ‘BERT’, these tokens are transformed into n-dimensional embeddings. Combining features from this process, a neural network ‘NN’ then predicts whether the message is Ham, Spam, or Smishing.

Algorithm 2: Message classification Algorithm

Inputs: Input Message String ('m')
Output: Message class (Ham, Spam, Smishing)

1.	Import Libraries: • Import necessary libraries including <code>re</code>, <code>string</code>, <code>transformers</code>, and <code>tensorflow</code>.
2.	Define Function to Extract Features (extract): • Use Algorithm 1
3.	Define Function to Preprocess Message (preprocess): • Input: <code>message</code> (m) • Output: <code>preprocessed_message</code> (m) • Remove emojis, punctuation, and convert the message to lowercase.
4.	Define Function to Tokenize Message (tokenize): • Input: <code>message</code> (m) • Output: <code>tokens</code> (list of strings) • Tokenize the preprocessed message using a BERT tokenizer.
5.	Define Function to Get BERT Embeddings (get_bert_embeddings): • Input: <code>tokens</code> (list of strings) • Output: <code>embeddings</code> (tensor) • Load a BERT model and get the embeddings for the input tokens.
6.	Define Function to Combine Features (combine_features): • Input: <code>embeddings</code>, <code>urls</code>, <code>phones</code>, <code>emails</code> • Output: <code>combined_features</code> (tensor) • Concatenate BERT embeddings with extracted features (URLs, phone numbers, emails).
7.	Define Function to Predict Class (predict_class): • Input: <code>features</code> (tensor) • Output: <code>class_predictions</code> (tensor) • Define a neural network model, compile it, and train it on labeled data. Then, predict the class labels using the trained model.

The process begins by extracting features from URLs, phone numbers, emails, and other identifying information from a message, indicating their presence due to their common use by attackers. Next, the message undergoes preprocessing: emojis and punctuation marks are removed, the text is converted to lowercase, and any URLs, phone numbers, and emails are extracted to standardize the content. Following this, the message is tokenized into individual units of meaning. Using transformers, particularly pre-trained BERT models, embeddings are constructed from the message to capture essential text characteristics. These embeddings are then combined with the initial extracted features to create a comprehensive input. Finally, this input is fed into two distinct neural network classifiers, one determining if the message is smishing and the other discerning if it's spam, enhancing the overall accuracy of the classification.

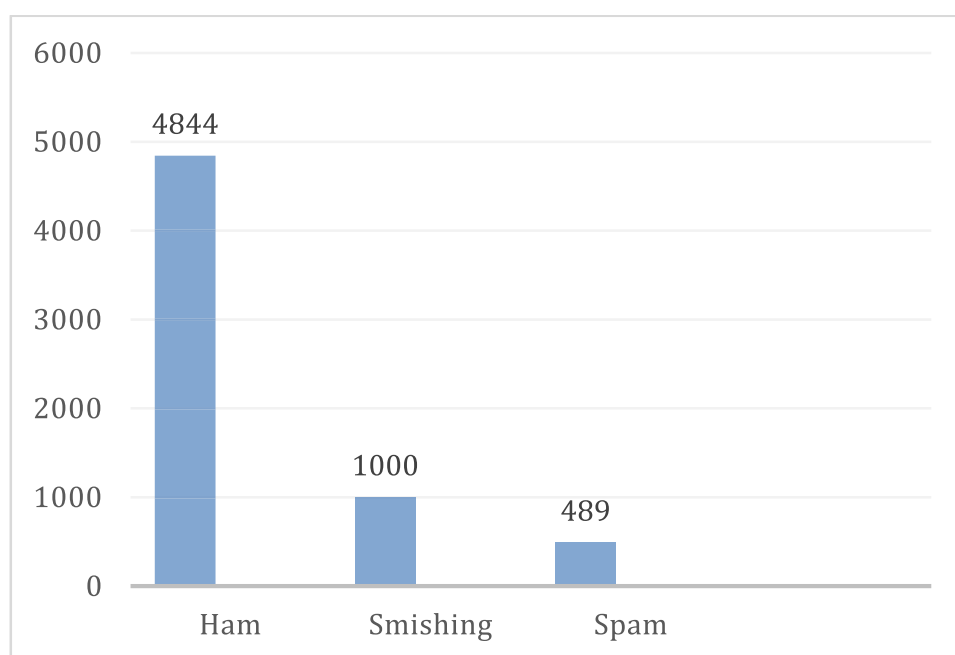
Experiment

In this section, we delved into the details of the datasets, examined the experimental environment, explored the evaluation metrics, and analyzed the results.

Details of Data Set

The initial dataset is sourced from the Almeida collection [21] and smishing images from Pinterest.com [22]. These Pinterest images were converted to text for the dataset. The total samples are 5971 including ham, spam, and smishing. To address the imbalance of only 638 smishing messages, we've expanded the collection to about 1000 smishing texts and are gathering more. Figure 7 shows the details of the dataset containing ham, spam, and smishing messages. Given the imbalance in the dataset, with more ham messages than smishing ones, adjustments are needed for accurate model outcomes. Solutions include:

Fig. 7 Class Distribution in Dataset: Ham, Spam, and Smishing



- a) **Oversampling and Undersampling:** In dealing with imbalanced datasets, two common strategies are oversampling and under-sampling. Oversampling involves duplicating or artificially creating new instances from the minority class. This helps achieve a balanced dataset and often aids in better classification. However, it comes with the caveat of potentially overfitting the model to the minority class due to excessive repetition. On the contrary, under-sampling reduces instances from the majority class. While this strategy makes the dataset more balanced and manageable, it may inadvertently discard significant patterns, leading to a potential loss in information.
- b) **Synthetic Data Generation:** Another approach to address data imbalance is synthetic data generation. This method involves creating new data instances that were not originally present in the dataset. One popular technique is back-translation, where a text is translated to a different language and then translated back to the original language, resulting in a slightly altered but contextually similar sentence. Another method, paraphrasing, involves rewording an existing piece of text to produce a new instance while retaining the same underlying meaning. These techniques can enhance the dataset's diversity, especially for the minority class.
- c) **Transfer Learning:** Transfer learning has gained prominence, especially in scenarios with limited or imbalanced data. In this approach, models like BERT and GPT, which have been pre-trained on vast and diverse datasets, are employed. Instead of training a model from

scratch, one can leverage these pre-trained models, adapting them to specific tasks by fine-tuning on the task-specific dataset. By doing so, the model inherits the knowledge from larger datasets, making it a powerful tool for tasks with limited data availability.

Experimental Environment

For our computational tasks, the system was backed by high-end Hardware specifications. We used an Intel Core i7-9700 K processor, which consists of 8 cores and operates at a base speed of 3.6 GHz. Assisting this CPU, we had 64GB DDR4 RAM, clocked at 3200 MHz. For rapid data access and storage, a 1 TB NVMe SSD was incorporated.

On the Software front, our platform of choice was Ubuntu 20.04 LTS, known for its stability and compatibility with deep learning frameworks. All our scripts and model training codes were written in Python 3.8. The experiment heavily relied on TensorFlow 2.5 for deep learning tasks, supplemented with the Transformers library for using the BERT model. Other essential libraries used for data preprocessing and model evaluation included Scikit-learn, NLTK, Pandas, and NumPy.

The core of the experiment involved working with a dataset of labeled SMS texts, categorized as smishing, ham, or spam. We began by preprocessing this data, which involved cleaning, normalizing, and tokenizing the text. Our main focus was on extracting relevant features from these messages using embeddings from TF-IDF and BERT.

Table 2 Accuracy and F1Score of classifiers evaluated (train: test = 2:1)

Classifier	Accuracy (in %)	Precision (in %)	Recall (in %)	F1-Score (in %)
Logistic Regression	93.46	93	93	93
SVM with RBF kernel	78.54	74	79	71
SVM with Polynomial kernel	90.57	84	91	87
Decision Tree	84.98	86	85	85.38
Random Forest	90.62	89	91	89.07
K-Nearest Neighbour	89.58	89	90	88.44
Gaussian NB	49.03	80	49	54.29
Gradient Boosting Classifier	91.74	91	91	90.82
AdaBoost Classifier	90.76	91	91	90.61
SVM with Sigmoid Kernel	79.30	72	79	75.44
XGBoost	92.33	92	92	91.70

Post-feature extraction, the models underwent training and validation phases. Performance was majorly benchmarked using metrics like accuracy, precision, recall, and F1-score.

Evaluation Metrics

Evaluation metrics are crucial in determining the performance of a model. They offer a quantitative measure of how

well the model performs against the true outcomes. Here's a detailed in below:

Accuracy This is a straightforward metric, representing the ratio of correctly predicted observations to the total observations. Mathematically, it is given by.

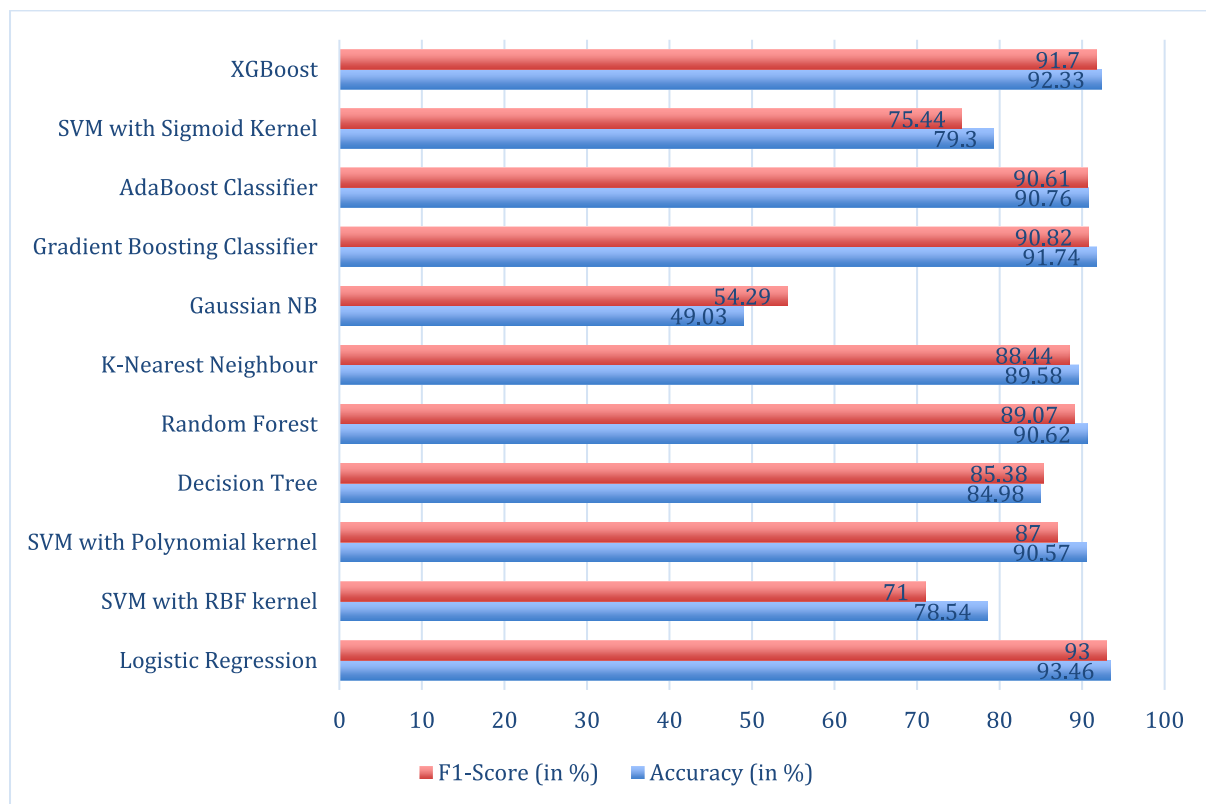
**Fig. 8** F1-score and Accuracy of ML based classifier for Train: Test = 2:1

Table 3 Accuracy and F1Score of classifiers evaluated (train: test = 80:20)

Classifier	Accuracy (in %)	Precision (in %)	Recall (in %)	F1-Score (in %)
Logistic Regression	94.00	94	94	93.78
SVM with RBF kernel	79.40	74	79	72.85
SVM with Polynomial kernel	91.00	85	91	87.47
Decision Tree	85.87	87	87	86.82
Random Forest	90.37	89	90	89.15
K-Nearest Neighbour	89.27	89	89	88.18
Gaussian NB	50.51	80	51	56.06
Gradient Boosting Classifier	91.32	90	91	90.65
AdaBoost Classifier	90.84	90	91	90.41
SVM with Sigmoid Kernel	78.14	71	78	74.61
XGBoost	92.42	92	92	92.02

$$\text{Accuracy} = \frac{\text{Number of True Predictions}}{\text{Total Number of Samples}} \quad (7)$$

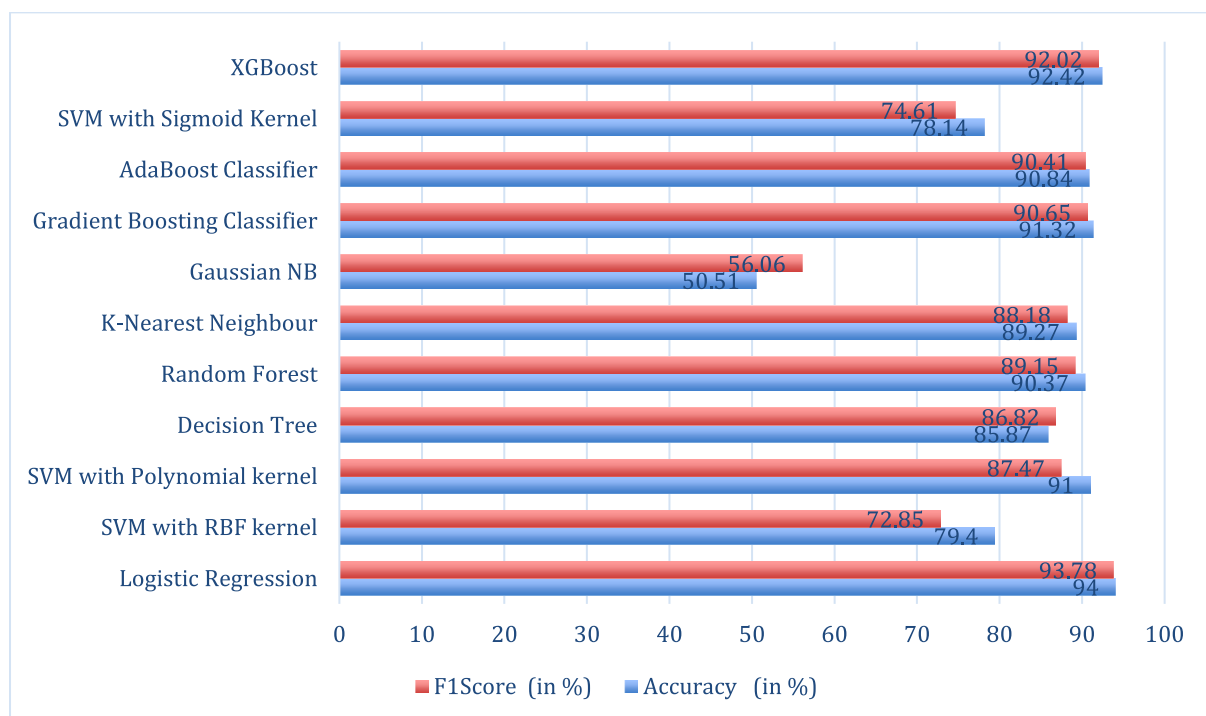
Precision Precision evaluates the number of correctly predicted positive outcomes against all the outcomes predicted as positive. It is given by.

$$\text{Precision} = \frac{\text{Number of True Positives}}{\text{Number of True Positives} + \text{Number of False Positives}} \quad (8)$$

Recall (or Sensitivity) This metric gauge the number of correctly predicted positive outcomes against all the actual positive outcomes. Its formula is.

$$\text{Recall} = \frac{\text{Number of True Positives}}{\text{Number of True Positives} + \text{Number of False Negatives}} \quad (9)$$

F1-Score The F1-Score is the harmonic mean of precision and recall. It offers a balance between the two metrics, especially when their values differ significantly.

**Fig. 9** F1-score and Accuracy of ML based classifier for Train: Test=4:1

$$F1 - Score = \frac{2 * Precision * Recall}{Precision + Recall} \quad (10)$$

True Positive Rate (TPR) Also known as Sensitivity or Recall, it represents the ratio of positive instances correctly predicted by the model. It can be represented as.

$$TPR = \frac{Number\ of\ True\ Positives}{Number\ of\ True\ Positives + Number\ of\ False\ Positives} \quad (11)$$

False Positive Rate (FPR) FPR measures the ratio of negative instances falsely predicted as positive by the model. It is calculated as.

$$FPR = \frac{Number\ of\ True\ Negatives}{Number\ of\ True\ Negatives + Number\ of\ False\ Negatives} \quad (12)$$

Area under the Curve (AUC) The ROC curve (Receiver Operating Characteristic) is a graphical representation that plots TPR against FPR at different classification thresholds. AUC represents the entire two-dimensional area underneath the entire ROC curve, and a higher AUC indicates a better model's performance.

The effectiveness of a classification model is calculated using the ROC curve. TPR (True Positive Rate) and FPR (False Positive Rate) are represented on the y-axis and x-axis, respectively, of the ROC curve. The model is better when the AUC is higher.

Results and Discussion

Table 2 showcases the comparative performances of different classifiers when the dataset is split in a 2:1 train-to-test ratio and Fig. 8 shows the bar graph for F1-score and accuracy of different classifiers. Notably, logistic regression stands out with an accuracy of 93.46%, and its precision, recall, and F1-score all converge at 93%. This is the highest accuracy among the models tested. In stark contrast, Gaussian NB lags with the lowest accuracy at 49.03%. However, it's worth noting its unexpectedly high precision of 80%. SVM classifiers, specifically with RBF and Sigmoid kernels, have comparable performance metrics, with accuracies teetering around the high 70s. Decision Trees and Random Forests strike a balance in their metrics, delivering accuracies between the mid-80s and low-90s percentiles.

Gradient Boosting and AdaBoost Classifiers shine with accuracies surpassing 90%. Their precision, recall, and F1-scores are impressively consistent. Similarly, XGBoost positions itself as a formidable model with an accuracy

of 92.33% and congruent precision, recall, and F1-score figures.

Table 3 showcases the performance metrics of various classifiers evaluated on a dataset split in an 80:20 train-test ratio and Fig. 9 illustrates the bar graph for F1-score and accuracy of different classifiers. Logistic Regression performed notably well, with an accuracy of 94.00% and an F1Score of 93.78%. The high precision of the LR classifier 94% shows that the accurate prediction of every class of message spam, ham, or smishing is very high. The SVM with RBF kernel and SVM with Sigmoid Kernel demonstrated a relatively lower accuracy of 79.40% and 78.14%, respectively. SVM with Polynomial kernel, Gradient Boosting Classifier, and XGBoost presented promising results with accuracy metrics close to or above 91%. Decision Tree, Random Forest, K-Nearest Neighbour, and AdaBoost Classifier showcased accuracy in the range of 85–91%. Gaussian NB had the lowest accuracy of 50.51% but maintained a precision of 80%. XGBoost stood out with an accuracy of 92.42% and an F1Score of 92.02%.

Comparison with State of art

Table 4 represents a comparison of various machine learning and deep learning models proposed by different authors for a specific task. Delany et al. [23] and Raj et al. [24] achieved accuracies of 94.63% and 97.50% using SVM and LSTM, respectively. The highest accuracy was achieved by Roy et al. at 99.44% with a 3CNN, Dropout, and 10-fold model. Two BERT-based models are also presented, one with a dataset ratio of 2:1 and another with a dataset ratio 4:1, with accuracies of 93.46% and 94% respectively. The metrics provided include accuracy, precision, recall, F1-Score, and AUC, though some entries are missing specific metrics. Table 4 shows the comparison of various existing smishing detection methods based upon accuracies and it shows our model accuracy is 93.46% and 94% for 2:1 and 4:1 ratio dataset respectively.

Conclusion and Future Directions

The proposed approach, rooted in state-of-the-art technologies, offers tangible benefits in tackling smishing threats. The model is built using embeddings such as TF-IDF, and BERT which produce the best results by recognizing contextual associations between words, and use of NLP strategies, such as tokenization and self-attention mechanisms for the precise categorization of messages into smishing, ham, or spam groups. Various Machine learning based classifiers have been used such as Logistic Regression, SVM with RBF kernel, SVM with Polynomial kernel, Decision Tree, Random Forest, K-Nearest Neighbour, Gaussian NB, Gradient

Table 4 Comparison of the proposed model with existing models

Author	Method	Accuracy (%)	Precision	Recall	F1-Score	Area Under the Curve (AUC)
Delany et al. [23]	SVM	94.63	NA	NA	NA	NA
Raj et al. [24]	LSTM	97.50	NA	NA	NA	NA
Narayan et al. [25]	Bayes + SVM	99.0	0.987	0.9900	0.9880	NA
Popovac et al. [26]	TF-IDF + CNN	98.40	NA	NA	0.9830	0.9550
Roy et al. [27]	3CNN + Dropout + 10-fold	99.44	0.9950 (weighted)	0.9955 (weighted)	0.9961 (weighted)	0.9770
Oswald et al. [2]	DistilBERT + SVM	98.07	0.9786 (weighted)	0.9787 (weighted)	0.9786 (weighted)	0.9483
Das et al. [19]	LSTM + DCNN + ROA	98.25	0.9872	0.9907	0.9889	NA
Seo et al. [20]	Deep Learning	99.59	NA	NA	0.9946	NA
Proposed Model (2:1)	BERT + multi-layer neural network	93.46	0.93 (weighted)	0.93 (weighted)	0.93 (weighted)	0.9326
Proposed Model (4:1)	BERT + multi-layer neural network	94.00	0.93 (weighted)	0.93 (weighted)	0.9378 (weighted)	0.9431

Boosting Classifier, Adaboost, SVM with Sigmoid Kernel and XGBoost. Based upon various evaluation criteria like Accuracy, Precision, Recall, and F1-score, the experimental results show that the Logistic Regression classifier outperforms all others with a highest accuracy of 94% as compared to other classifiers for both Train: Test ratios of 2:1 and 4:1. Overall, the model is robust and can accurately classify SMS as smishing, ham or spam. However, the existing model has various future scopes.

In the future, the work could be extended to analyse various social networking platform-based messages such as WhatsApp, Twitter, Facebook, and Instagram etc. And also try to collect other language-based datasets to develop a multi-lingual based smishing prevention system.

Author Contributions The manuscript was written by Ankit Kumar Jain and Kamaljeet Kaur and reviewed by all other co-authors. The main process flow was formerly designed by Naveen Kumar Gupta. and Ankit Khare. Implementation work has been done by Kamaljeet kaur.

Funding No funding was received during the preparation of this manuscript.

Data Availability Data will be available on request.

Declarations

Conflict of interest The authors have no relevant financial or non-financial interests to disclose.

Research Involving Human or Animal Not Applicable.

Informed Consent Not Applicable.

References

- Mishra S, Soni D. Implementation of ‘Smishing detector’: an efficient model for Smishing Detection using neural network. *SN Comput Sci*. 2022;3(3):1–13. <https://doi.org/10.1007/s42979-022-01078-0>.
- Oswald C, Simon SE, Bhattacharya A. SpotSpam: Intention Analysis-driven SMS spam detection using BERT Embeddings. *ACM Trans Web*. 2022 <https://doi.org/10.1145/3538491>.
- Tatango. 90% of SMS Marketing Messages Read in 3 Minutes, *tatango*, 2020. <https://www.tatango.com/blog/90-of-text-messages-are-read-within-3-minutes/> (accessed Dec. 24, 2023).
- Pemberton C. Tap Into The Marketing Power of SMS, *Gartner*, 2016. <https://www.gartner.com/en/marketing/insights/articles/tap-into-the-marketing-power-of-sms> (accessed Dec. 12, 2023).
- Akande ON, Akande HB, Kayode AA, Adeyinka AA, Olaiya F, Oluwadara G. Development of a Real Time Smishing Detection Mobile Application using rule based techniques. *Procedia Comput Sci*. 2021;199:95–102. <https://doi.org/10.1016/j.procs.2022.01.012>.
- Ghourabi A, Alohal M. Enhancing Spam Message Classification and Detection Using Transformer-Based Embedding and Ensemble Learning. *Sensors*. 2023;23(8):1–17. <https://doi.org/10.3390/s23083861>.
- Smitha C. A survey paper on smishing prediction system. *Int J Res Eng Sci*. 2023;11(4):470–3. <https://www.ijres.org/papers/Volume-11/Issue-4/1104470473.pdf>.
- Njuguna DN, Kamau J, Kaburu D. A Review of Smishing Attacks Mitigation Strategies. *Int J Comput Inf Technol*. 2022. <https://doi.org/10.24203/ijcit.v1i1.201>.
- Atera, SmiShing. 2024. <https://www.atera.com/glossary/smishing/#:~:text=An SMS message on average,leaning on mobile-based attacks.> (accessed Jun. 12, 2024).
- Jovanovic A. 10 Facts + Stats on Smishing (SMS Phishing) in 2024, *Safety Detectives*, 2024. <https://www.safetydetectives.com/blog/what-is-smishing-sms-phishing-facts/> (accessed Aug. 08, 2024).
- UKWDA. New banking phishing scam by letter for Halifax bank customer details, *Web Growth Consulting*. <https://webgrowth.co.uk/new-banking-phishing-scam-by-letter-for-halifax-bank-customer-details/> (accessed Dec. 12, 2023).

12. Mishra S, Soni D. Smishing detector: a security model to detect smishing through SMS content analysis and URL behavior analysis. *Futur Gener Comput Syst*. 2020;108:803–15. <https://doi.org/10.1016/j.future.2020.03.021>.
13. Jain AK, Gupta BB, Kaur K, Bhutani P, Alhalabi W, Almomani A. A content and URL analysis-based efficient approach to detect smishing SMS in intelligent systems. *Int J Intell Syst*. 2022;37(12):11117–41. <https://doi.org/10.1002/int.23035>.
14. Jain AK, Gupta BB. Rule-based Framework for detection of smishing messages in Mobile Environment. *Procedia Comput Sci*. 2018;125:617–23. <https://doi.org/10.1016/j.procs.2017.12.079>.
15. Salman M, Ikram M, Kaafar MA. Investigating Evasive Techniques in SMS Spam Filtering: A Comparative Analysis of Machine Learning Models. *IEEE Access*. 2023;12:24306–24. <https://doi.org/10.1109/ACCESS.2024.3364671>.
16. Choudhary N, Jain AK. Towards filtering of SMS spam messages using machine learning based technique. *Adv Inf Comput Res*. 2017;1:18–30.
17. Akande ON, et al. SMSPROTECT: an automatic smishing detection mobile application. *ICT Express*. 2023;9(2):168–76. <https://doi.org/10.1016/j.icte.2022.05.009>.
18. Santosh Kumar Birthriya PA, Jain AK. A Comprehensive Survey of Social Engineering attacks: taxonomy of attacks, Prevention, and mitigation strategies. *J Appl Secur Res*. 2024;0(0):1–49. <https://doi.org/10.1080/19361610.2024.2372986>.
19. Das L, Ahuja L, Pandey A. A novel deep learning model-based optimization algorithm for text message spam detection. *J Supercomput*. 2024;80(12):17823–48. <https://doi.org/10.1007/s11227-024-06148-z>.
20. Seo JW, et al. On-Device Smishing Classifier Resistant to Text Evasion Attack. *IEEE Access*. 2024;12:4762–79.
21. Almeida SMSS, Collection, Dataset. *Kaggle*, 2024. <https://www.kaggle.com/datasets/uciml/sms-spam-collection-dataset> (accessed Jan. 20, 2024).
22. Edu S, Lee J. SMiShing dataset, *Pinterest*, 2024. <https://www.pinterest.com.au/seceduau/smishing-dataset/> (accessed Jan. 12, 2024).
23. Delany SJ, Buckley M, Greene D. SMS spam filtering: methods and data. *Expert Syst Appl*. 2012;39(10):9899–908. <https://doi.org/10.1016/j.eswa.2012.02.053>.
24. Raj H, Weihong Y, Banbharni SK, Dino SP. LSTM based short message service (SMS) modeling for spam classification. *ACM Int Conf Proceeding Ser*. 2018. <https://doi.org/10.1145/3231884.3231895>.
25. Narayan A, Saxena P. The curse of 140 characters: Evaluating the efficacy of SMS spam detection on Android. *Proc ACM Conf Comput Commun Secur*. 2013. <https://doi.org/10.1145/2516760.2516772>.
26. Popovac M, Karanovic M, Sladojevic S, Arsenovic M, Anderla A. Convolutional Neural Network Based SMS Spam Detection, 2018 *26th Telecommun. Forum, TELFOR 2018 - Proc.*, pp. 1–4, 2018, <https://doi.org/10.1109/TELFOR.2018.8611916>.
27. Roy PK, Singh JP, Banerjee S. Deep learning to filter SMS spam. *Futur Gener Comput Syst*. 2020;102:524–33. <https://doi.org/10.1016/j.future.2019.09.001>.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Terms and Conditions

Springer Nature journal content, brought to you courtesy of Springer Nature Customer Service Center GmbH (“Springer Nature”).

Springer Nature supports a reasonable amount of sharing of research papers by authors, subscribers and authorised users (“Users”), for small-scale personal, non-commercial use provided that all copyright, trade and service marks and other proprietary notices are maintained. By accessing, sharing, receiving or otherwise using the Springer Nature journal content you agree to these terms of use (“Terms”). For these purposes, Springer Nature considers academic use (by researchers and students) to be non-commercial.

These Terms are supplementary and will apply in addition to any applicable website terms and conditions, a relevant site licence or a personal subscription. These Terms will prevail over any conflict or ambiguity with regards to the relevant terms, a site licence or a personal subscription (to the extent of the conflict or ambiguity only). For Creative Commons-licensed articles, the terms of the Creative Commons license used will apply.

We collect and use personal data to provide access to the Springer Nature journal content. We may also use these personal data internally within ResearchGate and Springer Nature and as agreed share it, in an anonymised way, for purposes of tracking, analysis and reporting. We will not otherwise disclose your personal data outside the ResearchGate or the Springer Nature group of companies unless we have your permission as detailed in the Privacy Policy.

While Users may use the Springer Nature journal content for small scale, personal non-commercial use, it is important to note that Users may not:

1. use such content for the purpose of providing other users with access on a regular or large scale basis or as a means to circumvent access control;
2. use such content where to do so would be considered a criminal or statutory offence in any jurisdiction, or gives rise to civil liability, or is otherwise unlawful;
3. falsely or misleadingly imply or suggest endorsement, approval, sponsorship, or association unless explicitly agreed to by Springer Nature in writing;
4. use bots or other automated methods to access the content or redirect messages
5. override any security feature or exclusionary protocol; or
6. share the content in order to create substitute for Springer Nature products or services or a systematic database of Springer Nature journal content.

In line with the restriction against commercial use, Springer Nature does not permit the creation of a product or service that creates revenue, royalties, rent or income from our content or its inclusion as part of a paid for service or for other commercial gain. Springer Nature journal content cannot be used for inter-library loans and librarians may not upload Springer Nature journal content on a large scale into their, or any other, institutional repository.

These terms of use are reviewed regularly and may be amended at any time. Springer Nature is not obligated to publish any information or content on this website and may remove it or features or functionality at our sole discretion, at any time with or without notice. Springer Nature may revoke this licence to you at any time and remove access to any copies of the Springer Nature journal content which have been saved.

To the fullest extent permitted by law, Springer Nature makes no warranties, representations or guarantees to Users, either express or implied with respect to the Springer nature journal content and all parties disclaim and waive any implied warranties or warranties imposed by law, including merchantability or fitness for any particular purpose.

Please note that these rights do not automatically extend to content, data or other material published by Springer Nature that may be licensed from third parties.

If you would like to use or distribute our Springer Nature journal content to a wider audience or on a regular basis or in any other manner not expressly permitted by these Terms, please contact Springer Nature at

onlineservice@springernature.com